

Alberta Energy Data Pipeline

Airflow-orchestrated ETL with a quarantine-based audit layer, idempotent loads, and dbt marts

Role: Data pipeline engineer

Python

Apache Airflow

PostgreSQL

dbt

Docker

GitHub Actions

Power BI

Built an Airflow-orchestrated ETL pipeline for Alberta Energy Regulator production data. A validation and audit layer quarantines bad records, loads are idempotent upserts, and a dbt analytics layer feeds Power BI. CI runs the whole pipeline twice against live PostgreSQL to prove re-runs are safe.

2x

CI Runs per Build

Whole pipeline run twice on live PostgreSQL to prove idempotency

~299x

Faster Extraction

53.3 s to 0.18 s, output verified identical

85%

Coverage Gate

Enforced by pytest in GitHub Actions

Problem

AER publishes production data as report-style Excel workbooks with merged cells and year columns. The format is hard for analysts to query or trust, and naive reloads either duplicate or wipe history.

Solution

A scheduled pipeline extracts, transforms, validates, and upserts records into PostgreSQL, quarantining anything invalid. dbt models the analytics layer, and Power BI reads it. Every run is audited.

Architecture

- Airflow DAG: optional source download, then crude oil and natural gas ETL in parallel
- Python extract / transform / validate / load modules
- Quality-issue tables and a pipeline_runs audit table
- PostgreSQL with natural-key unique indexes backing chunked upserts
- dbt staging views, fct_production, and summary marts with data tests
- Docker Compose (Airflow + PostgreSQL 16), Power BI dashboard

Highlights

- Validation quarantines nulls, bad dates, negative volumes, and duplicates into quality-issue tables
- Per-run audit records track row counts and error rates
- Idempotent incremental loading: chunked upserts against natural-key unique indexes
- dbt staging views, a unified fact table, and summary marts, all enforced by dbt data tests
- Two-stage CI: unit tests, then an integration job that runs the pipeline twice and executes dbt build
- Extraction cut from 53.3 s to 0.18 s by removing a duplicate parse and switching to a Rust-backed reader

Challenges and Tradeoffs

- Challenge: Re-running a load could duplicate rows or erase history. Change: Chunked upserts on natural keys, proven in CI by running the pipeline twice.

- Challenge: Deleting bad rows hides data problems. Change: Quarantine them into quality-issue tables and record error rates per run.
- Challenge: Extraction took 53 s per run. Change: Removed a duplicate workbook parse and switched to python-calamine, getting 0.18 s with identical output.
- Tradeoff: Requiring the fast reader vs portability. Decision: Fall back to pandas' default engine if it isn't installed.

Outcomes

- A repeatable, observable pipeline with an audit trail for every run
- Re-runs are provably safe, so backfills and retries are low-risk
- Benchmark scripts included so the performance claims are reproducible

Repository: <https://github.com/Jaskeeratr/alberta-energy-pipeline> · Full case study: <https://jaskeerat-portfolio.vercel.app/projects/alberta-energy-data-pipeline>